

JavaScript + MySQL

Cadastro de nome e e-mail

Aula prática com Node.js, Express e banco de dados MySQL

Um pequeno sistema de cadastro

- Uma tabela de banco de dados chamada usuarios.
- Uma API em Node.js que grava e consulta os dados.
- Uma página HTML para cadastrar e listar pessoas.

Fluxo da aplicação

Navegador

↓ envia nome e e-mail

Servidor Node.js

↓ executa SQL

MySQL

MySQL

Sistema que armazena dados em tabelas.

Banco de dados

Conjunto organizado de tabelas.

Tabela

Estrutura com colunas e linhas.

Node.js

Ambiente que permite executar JavaScript fora do navegador.

API

Endereços do servidor que recebem e devolvem informações.

Instale estes programas antes de escrever código

1. Node.js

Baixe a versão LTS em nodejs.org. Ela inclui o npm, usado para instalar bibliotecas.

2. MySQL Server

Instale o MySQL Community Server e guarde a senha do usuário root.

3. MySQL Workbench

Interface gráfica para criar banco, tabelas e executar comandos SQL.

4. VS Code

Editor de código recomendado para criar os arquivos do projeto.

Estrutura de pastas e arquivos

05

Crie uma pasta chamada cadastro-mysql. Dentro dela, crie os itens abaixo.

```
cadastro/  
  public/  
    index.html    ← tela do usuário  
    style.css     ← aparência da tela  
    script.js     ← ações no navegador  
  .env           ← dados de conexão do MySQL  
  database.js    ← conexão com o banco  
  server.js      ← servidor e rotas da API  
  package.json   ← bibliotecas do projeto
```

Atenção

Não publique o arquivo .env em repositórios públicos. Ele contém a senha do banco.

No MySQL Workbench, abra uma nova aba SQL, cole o script e clique no ícone de raio.

```
CREATE DATABASE cadastro_db;

USE cadastro_db;

CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

O que cada coluna significa

- id: identificador único.
- nome: texto obrigatório.
- email: texto obrigatório e sem repetição.
- criado_em: data e hora do cadastro.

Abra o terminal na pasta cadastro-mysql e execute os comandos.

```
npm init -y  
  
npm install express mysql2 dotenv  
  
npm install --save-dev nodemon
```

O que foi instalado

- express: cria o servidor web.
- mysql2: conecta ao MySQL.
- dotenv: lê variáveis do .env.
- nodemon: reinicia o servidor ao salvar.

Crie o arquivo .env na raiz da pasta cadastro-mysql.

```
DB_HOST=localhost  
DB_USER=root  
DB_PASSWORD=sua_senha_do_mysql  
DB_NAME=cadastro_db  
PORT=3000
```

Por que usar .env?

Assim, a senha fica separada do código. Troque “sua_senha_do_mysql” pela senha criada durante a instalação.

Este arquivo abre uma conexão reutilizável com o MySQL.

```
// database.js
require('dotenv').config();
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true
});

module.exports = pool;
```

pool guarda conexões que podem ser reutilizadas. process.env lê os dados do arquivo .env.

Arquivo server.js: configuração

10

O servidor recebe pedidos do navegador e responde.

```
// server.js
const express = require('express');
const path = require('path');
const pool = require('./database');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));
```

`express.json()` permite receber dados em JSON.

`express.static()` libera a pasta public para o navegador.

Arquivo server.js: rota de cadastro

11

Adicione este código abaixo da configuração anterior.

```
app.post('/usuarios', async (req, res) => {
  const { nome, email } = req.body;

  if (!nome || !email) {
    return res.status(400).json({ erro: 'Informe nome e e-mail.' });
  }

  try {
    const sql = 'INSERT INTO usuarios (nome, email) VALUES (?, ?)';
    const [resultado] = await pool.execute(sql, [nome, email]);
    res.status(201).json({ id: resultado.insertId, nome, email });
  } catch (erro) {
    res.status(500).json({ erro: 'Não foi possível cadastrar.' });
  }
});
```

Os símbolos ? evitam misturar diretamente os dados do usuário com o comando SQL.

```
app.get('/usuarios', async (req, res) => {
  try {
    const [usuarios] = await pool.execute(
      'SELECT id, nome, email, criado_em FROM usuarios ORDER BY id DESC'
    );
    res.json(usuarios);
  } catch (erro) {
    res.status(500).json({ erro: 'Não foi possível consultar.' });
  }
});

app.listen(PORT, () => {
  console.log(`Servidor em http://localhost:${PORT}`);
});
```

Rota GET

Quando o navegador acessa /usuarios, o servidor devolve a lista em JSON.

Salve server.js antes de continuar.

Crie a tela com campos para nome e e-mail.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cadastro de usuários</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main>
    <h1>Cadastro de usuários</h1>
    <form id="formulario">
      <input id="nome" placeholder="Digite o nome" required>
      <input id="email" type="email" placeholder="Digite o e-mail" required>
      <button type="submit">Cadastrar</button>
    </form>
    <p id="mensagem"></p><ul id="lista"></ul>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

Este JavaScript envia o formulário à API e depois atualiza a lista.

```
const form = document.querySelector('#formulario');
const mensagem = document.querySelector('#mensagem');

async function carregarUsuarios() {
  const resposta = await fetch('/usuarios');
  const usuarios = await resposta.json();
  document.querySelector('#lista').innerHTML = usuarios
    .map(u => `<li>${u.nome} - ${u.email}</li>`).join('');
}

form.addEventListener('submit', async (evento) => {
  evento.preventDefault();
  const nome = document.querySelector('#nome').value;
  const email = document.querySelector('#email').value;
  const resposta = await fetch('/usuarios', {
    method: 'POST', headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({ nome, email })
  });
  mensagem.textContent = resposta.ok ? 'Cadastro realizado!' : 'Erro ao cadastrar.';
  if (resposta.ok) { form.reset(); carregarUsuarios(); }
});

carregarUsuarios();
```

Use este CSS simples para melhorar a leitura da tela.

```
body { font-family: Arial, sans-serif; background: #eaf4fa; }
main { max-width: 600px; margin: 40px auto; background: white; padding: 28px; }
input, button { width: 100%; box-sizing: border-box; padding: 12px; margin-top: 12px; }
button { background: #1261a0; color: white; border: 0; cursor: pointer; }
li { margin: 10px 0; }
```

Salve este arquivo dentro da pasta public. O index.html já possui o link para carregá-lo.

No terminal, ainda dentro da pasta cadastro-mysql, execute:

```
npx nodemon server.js
```

1. Veja no terminal: “Servidor em http://localhost:3000”.
2. Abra o navegador e acesse http://localhost:3000.
3. Informe nome e e-mail e clique em Cadastrar.
4. Abra o MySQL Workbench e execute: `SELECT * FROM usuarios;`

Access denied for user root

Confira DB_USER e DB_PASSWORD no .env.

Unknown database cadastro_db

Execute novamente CREATE DATABASE cadastro_db;

Cannot GET /usuarios

Use o endereço `http://localhost:3000/usuarios`.

E-mail já existe

A tabela não permite repetir e-mail. Teste outro e-mail.

Servidor não inicia

Verifique se executou `npm install` e se está na pasta certa.

Vou verificar pessoalmente os produtos de software

Resultado esperado: você deve conseguir cadastrar pessoas pela página e encontrar os mesmos dados na tabela usuarios do MySQL.